



Unidad 4 / Escenario 7

Lectura fundamental

Recopilación de datos y estimaciones de tiempo en proyectos de *software*

Contenido

- 1 Recopilación de datos de buena calidad
- 2 Calculando el tamaño del proyecto/producto
- 3 Calculando el tiempo y la agenda de un proyecto

Palabras clave:

Estimación, líneas de código, tamaño de componentes, método probe, proxies.

Introducción

Bienvenido a la primera parte de la unidad de gestión de calidad en proyectos del curso TSP. Como recordará, la ingeniería de *software* busca la mejora en la calidad de los productos de *software* desde su planeación e implementación, para reducir la cantidad de defectos en el producto terminado; esto a su vez reduce el tiempo y el costo del mantenimiento de ese producto. Ya hemos visto cómo producir un plan para nuestros proyectos que nos permita hacer seguimiento a nuestro progreso y a la calidad de nuestro código, pero no hemos entrado en detalle cómo hacer las cuentas iniciales. En el módulo anterior, mencionamos que muchas veces cuando los planes necesitan ajustarse, se puede deber a una mala estimación por parte del equipo.

Entonces, ¿cómo realizar una buena estimación del tamaño de un proyecto o de cuánto tardará en completarse?

Bueno, eso es lo que veremos en este módulo; aprenderemos la importancia de capturar información de forma correcta y cómo usar esta información para estimar tanto el tamaño de nuestro proyecto como la agenda del mismo. ¡Comencemos!

1. Recopilación de datos de buena calidad

Como recordará, en el escenario anterior vimos la importancia de tener metas claras y cuantificables. Ahora, veremos cómo obtener los datos para medir nuestro progreso y tomar decisiones respecto a diferentes aspectos del proyecto. Primero, aclaremos que tener “algunos” datos, así no sean buenos, es mejor que no tener datos (exceptuando el caso donde se sepa que están completamente erróneos). Idealmente, los datos sobre el progreso de nuestras metas se recopilarán en tiempo real, a medida que se avanza con el proyecto, y estos son guardados para futuros proyectos (o, incluso, relanzamientos del mismo) a modo de realizar mejores estimaciones.

Recolectar la información de forma manual puede ser bastante engorroso. Una de las medidas de tamaño más comunes para proyectos de *software* son las líneas de código (LOC). A partir de esta medida, podemos estimar tiempos de trabajo y la calidad de nuestro producto (cosa que veremos en el siguiente escenario), entre otras cosas. El contar estas líneas a mano tardaría mucho tiempo y probablemente no sea exacto, ¿qué hacer entonces? Podemos utilizar una herramienta previamente diseñada para contar las líneas de código, una rápida búsqueda de google da como resultado varias herramientas libres.

Una vez tengamos datos históricos de al menos otros tres proyectos similares, podremos comenzar a sacar estimaciones de forma relativamente acertada. Aun si nuestras primeras estimaciones no se acercan mucho, estas nos ayudarán a calcular nuevos y mejores valores.

Uno de los métodos de solución de problemas que más enseñan en el mundo de sistemas es el de tomar un problema y dividirlo en una serie de problemas más pequeños, antes de solucionarlos; de forma similar, la estimación de un producto de *software* será más acertada si se toma la suma de las estimaciones hechas para cada uno de sus componentes, que si se toma el proyecto como una única unidad grande. Por ejemplo, asumiendo una arquitectura Modelo-Vista-Controlador (MVC), tomaríamos la suma de LOC en todos los Controladores, todas las Vistas, y todos los archivos que conforman el Modelo para estimar el tamaño del proyecto final (nótese que podemos descomponer esto hasta llegar a LOC por método, lo veremos más despacio luego).

2. Calculando el tamaño del proyecto/producto

El tamaño de un proyecto y la cantidad de tiempo que se invierte en este suelen estar correlacionados. A fin de cuentas, los proyectos más simples y rápidos de terminar suelen ser aquellos de menor tamaño. No tiene mucho sentido pensar que un mismo equipo tarda lo mismo para un código que abarque 100 mil LOC, que en uno que abarque 500 mil.

Gracias a esto, se pueden crear planes basados en las estimaciones de tamaño y de esfuerzo conocidas para un equipo particular. También se puede utilizar esta información para determinar el esfuerzo requerido y la calidad del producto (siempre que tengan datos de defectos relacionados con el tamaño).

Medir el tamaño de un proyecto de *software* requiere de una medida apropiada; por ejemplo, para medir el tamaño de una casa utilizamos el área de esta, en lugar de algo como la cantidad de personas que pueden habitarla. Aunque ambas dan una medida de tamaño, ambas medidas cumplen un propósito diferente: un cliente puede que no pueda comprender la diferencia entre una casa de 90 metros cuadrados y otra de 150 metros cuadrados, pero comprenderá la diferencia entre una donde pueden vivir 2 personas a una donde pueden vivir 3 o 4.

Como ya habíamos mencionado antes, las LOC son una de las medidas más comunes para determinar el tamaño de un proyecto. De nada sirve saber que un proyecto tiene 40 clases diferentes, ya que esta medida por sí sola no me indica la complejidad o tamaño de estas (aunque podría suponer que es más complejo que un proyecto con 6 clases). Las líneas de código, sin embargo, relatan cuánto esfuerzo se invirtió en producir el proyecto. No necesariamente se sabe

cuántas clases tiene el proyecto, pero se puede inferir la complejidad del mismo.

Las medidas del tamaño de un proyecto pueden separarse en dos categorías:

- **Medidas físicas:** en términos simples, estas medidas son las que, sin importar quién las cuente (aunque deben contarse con el mismo método), las LOC de un proyecto o uno de sus componentes en bruto son una medida física
- **Medidas lógicas:** estas nos dan información en términos de agrupaciones de entidades físicas; por ejemplo, las LOC por método son una medida lógica. Las medidas lógicas no necesariamente corresponden a la medida física de la misma entidad, dependiendo del estándar definido para realizar la medida lógica. En *software* las medidas lógicas de líneas de código se deben determinar de acuerdo con un estándar de conteo que sea apropiado para el lenguaje de programación, con lo cual se intenta contar la cantidad de líneas ejecutables. Por ejemplo, la cantidad de líneas de código que terminan en “;” en el lenguaje de programación C, sería una medida lógica del tamaño del código.

Las medidas lógicas no siempre se pueden contar de forma automática y dependen del lenguaje de programación utilizado, lo que lleva a algunas dificultades a la hora de estimar su tamaño. Por ejemplo, una aplicación con estructura MVC puede utilizar varios lenguajes de programación: JavaScript para la parte de los controladores y en algunas partes de las vistas y PHP en el modelo, esto llevaría a dos medidas lógicas diferentes para diferentes partes del proyecto.

Por otra parte, las LOC no necesariamente están correlacionadas con la funcionalidad de un programa. Un programador experimentado puede resolver un problema con relativamente pocas líneas de código, mientras un novato puede hacerlo de forma muy ineficiente.

2.1. Diferentes medidas de tamaño

Para propósitos del TSP cualquier método de medición está bien siempre que se mantenga consistencia. Debido a la prevalencia de LOC, las usaremos en este texto.

Ahora, para medir el tamaño de un proyecto hay varios datos que se deben de recopilar:

- **Tamaño base:** este es el tamaño de un producto antes de modificarlo. Es el código con el que se comienza a trabajar. Esta medida suele ser usada en reuniones de relanzamiento o a la hora de desarrollar una nueva versión de un *software*.
- **Tamaño del producto agregado:** esta medida corresponde a la cantidad de LOC que ha

agregado al producto. No cuenta las líneas de código original que fueron modificadas.

- **Tamaño del producto modificado:** cuenta las líneas del código original que fueron modificadas, normalmente para corregir errores. No cuenta las líneas borradas.
- **Tamaño del producto agregado y modificado:** la suma de las dos medidas anteriores.
- **Tamaño del producto borrado:** cuenta las LOC que fueron eliminadas del código original.
- **Tamaño del producto reutilizable:** cuenta la cantidad de líneas de las librerías reutilizables que tenga el programa.
- **Tamaño del producto reutilizado:** cuenta la cantidad de líneas que provienen de librerías reutilizables y que no han sido modificadas. No cuenta líneas de código del código original que no hayan sido modificadas.
- **Tamaño del producto reutilizable nuevo:** cuenta las líneas agregadas a las librerías reutilizables. Ya se considera dentro del producto agregado.
- **Tamaño total:** cuenta todas las LOC, independiente de su fuente.

Es decir, para estimar el crecimiento de nuestro producto en una fase, estimamos el tamaño del producto agregado y modificado. Para estimar el tamaño total del producto, operamos sobre el producto base, esto nos indica el tamaño total para nuestro producto, y no cuenta las líneas modificadas (ya que se encuentran en el tamaño base y podríamos considerar una línea modificada como una línea borrada y otra agregada).

2.2. Estimación del tamaño de un proyecto

Bueno, ¿y entonces como estimamos el tamaño de un proyecto?

Lo primero que debemos de hacer es mirar si contamos con datos históricos, idealmente de proyectos similares al actual. Luego, utilizaremos el método de estimación basada en proxies (PROxy Based Estimation – PROBE) para estimar el tamaño de nuestro proyecto.

PROBE toma partes de metodologías externas y las combina; por un lado, está la estimación difusa, que le asigna términos difusos a los diferentes componentes del código para ayudar a determinar su tamaño y, por otro, está la estimación por medio para agregar los tamaños de estos componentes.

Supongamos que tenemos una casa con cuartos pequeños, medianos y grandes. Con base en combinar cuartos de diferentes tamaños, podemos intuir el tamaño de la casa como pequeña, mediana o grande, según nuestra definición de cada término. Para un arquitecto el saber qué tipo de cuarto es (baño, habitación, closet, cocina, etc.) y su tamaño (pequeño, mediano o grande), le permite estimar el área de la casa con base en su conocimiento previo del área de cada cuarto.

En TSP utilizamos PROBE que combina estos dos procesos, y luego utiliza regresión lineal para calcular el tamaño. Para la parte difusa del proceso, PROBE tiene cinco tamaños:

Muy Pequeño – Pequeño – Mediano – Grande – Muy Grande

Very Small – Small – Medium – Big – Very Big

Todos los componentes deben asignarse a uno de estos tamaños. Al conocerse datos históricos que permitan asignar un valor de LOC a cada rango, podemos estimar el tamaño de cada componente y luego calcular el tamaño inicial de nuestro proyecto al sumarlos. Sin embargo, no tenemos que parar ahí, también podemos dividir los componentes según su tipo:

- Lógica – control de flujo
- Entrada y salida (I/O)
- Texto
- Cálculos
- Datos
- Preparación

Y a cada uno de estos, asignarle diferentes valores de LOC según su tamaño. La suma de todos estos pequeños tamaños ayudará a estimar más precisamente el tamaño final del proyecto, y los valores de los intervalos los determinaremos con la ayuda de algo de estadística. A partir de los datos históricos que tengamos, sacaremos una distribución normal de estos valores; esto teniendo en cuenta que la media de la distribución corresponde, naturalmente, al intervalo de tamaño Mediano, luego una desviación estándar por encima se encuentra el tamaño Grande, y dos por encima se encuentra el tamaño Muy Grande. Lo análogo es cierto para los tamaños Pequeño y Muy Pequeño.

Ahora, con los datos históricos y el valor de cada uno de nuestros intervalos, podemos estimar el tamaño de nuestro proyecto con regresión lineal, esta parte es un poco densa, así que vamos despacio y con calma.

Llamemos a la estimación inicial que hemos hecho X, y Y a lo que sería el tamaño “real” estimado de nuestro proyecto (esto porque para toda estimación X existe una realidad Y. Queremos predecir el valor real Y; el cual compararemos con el valor real de nuestro proyecto una vez lo hayamos terminado).

Con los datos históricos de tamaños estimados (que llamaremos X_i) y los tamaños reales (que llamaremos Y_i) de código de al menos tres proyectos, podemos generar una estimación mucho más acertada de nuestro tamaño inicial del proyecto.

La regresión lineal nos dice que el tamaño “real” de nuestro proyecto, Y, está dado en función de nuestro tamaño estimado, B, de la siguiente forma:

$$Y = AX + B$$

Donde A esta dado por:

$$A = \frac{\sum_{i=0}^n (x_i * y_i - n * x_{promedio} * y_{promedio})}{\sum_{i=0}^n (x_i^2 - n * x_{promedio}^2)}$$

Y B está dado por:

$$B = y_{promedio} - (A * x_{promedio})$$

Es decir, el valor de A es calculado al tomar la sumatoria de la resta entre los productos, entre X y Y (tamaños estimado y real, respectivamente) de los proyectos viejos, menos el producto de X y Y promedios, y dividirlo entre la sumatoria de la resta entre cada valor de X al cuadrado y el producto de la cantidad de datos (N) y el valor de x promedio al cuadrado.

B, por su parte, se obtiene al encontrar la diferencia entre el valor de Y promedio y el producto entre A y X promedio.

Como X es nuestra estimación inicial, podemos encontrar un resultado “real” para nuestro proyecto al encontrar los valores de A y B. Este nuevo resultado, Y, será el que utilizaremos dentro de todos los planes como el tamaño estimado del proyecto.

Para el relanzamiento de un proyecto, X y Y representarían la cantidad de LOC que agregaríamos (agregadas + modificadas) estimadas y real, respectivamente.

Vale la pena notar que podemos y deberíamos hacer este proceso para los diferentes componentes, para un mejor resultado. Tendríamos muchos más puntos de comparación individuales para cada componente, y al sumar todos estos podríamos obtener una estimación bastante buena de nuestro X , para luego encontrar nuestro Y . Ya con datos del tamaño del proyecto en términos de LOC, podemos usar nuestros datos históricos para estimar cuánto tiempo tardaremos en realizar el proyecto. Esta estimación se puede revisar a futuro ajustándola a las capacidades del equipo y teniendo en cuenta la cantidad de horas de trabajo con las que se disponen semanalmente.

3. Calculando el tiempo y la agenda de un proyecto

El PV se determina al estimar la totalidad de horas de trabajo que se necesitarán para terminar el proyecto, y luego calculando el porcentaje que una tarea en particular representa de este total. De 1000 horas de trabajo, una tarea que se estima tardará 32 horas tiene un PV de 3,2, independientemente de cuantas horas de hecho se tarde en completar.

Aun si esta tarea tarde 45 horas, sólo aportará un EV de 3,2 al proyecto.

El EV se halla al determinar qué tareas se han completado en cualquier momento dado del proyecto. Cada tarea terminada suma un EV equivalente al PV de esa tarea, y el proyecto como tal tiene valores PV totales (o acumulativos hasta el día actual) y valores semanales (esta semana se espera obtener un EV de 6, pero el total que deberíamos haber acumulado es 15).

Si el EV total acumulado se acerca al PV acumulado al día, el proyecto no se encuentra atrasado significativamente. Un EV superior al PV acumulado al día indica que están adelantados, y un EV menor indica que están atrasados.

Si el EV acumulado de la semana se acerca al PV de la semana, indica que esa semana estarían trabajando al ritmo adecuado (y posiblemente desatrasándose).

En el escenario anterior se hizo referencia al valor obtenido (EV) y al valor planeado (PV) como dos medidas que podíamos utilizar para medir nuestro progreso en un proyecto. Recordemos brevemente:

Ya con la cantidad de horas de trabajo disponibles por semana, se calculan los valores de PV para cada semana del proyecto y los tiempos de las tareas. Si tenemos datos históricos que muestren una relación entre LOC y el tiempo requerido para realizarlas, podemos aplicar el método descrito en

el punto anterior nuevamente, para determinar cuánto tiempo tardará realizar una tarea de LOC. Durante el lanzamiento, al balancear la carga de trabajo cuidadosamente se puede estimar la agenda del proyecto, incluida su fecha de entrega.

Con base en las horas de trabajo disponibles por semana y en el orden que se desean (y se pueden) completar las tareas, vamos agregando semanas a la agenda de una en una, hasta que llegamos al final del proyecto.

Referencias bibliográficas

Ampuero, M., & López Trujillo, Y. (2010). *Creando un profesional con disciplina en el proceso de desarrollo de software*. Ingeniería Industrial, 27(1), 4 pág. Recuperado de <http://rii.cujae.edu.cu/index.php/revistaind/article/view/98/77>

Humphrey, W. S. (2000a). *The Personal Software Process SM (PSP SM)*, (November).

Humphrey, W. S. (2000b). *The Team Software Process SM (TSP SM)*, (November).

Humphrey, W. S., Chick, T. A., & Pomeroy-huff, M. (2010). *Team Software Process SM (TSP SM) Body of Knowledge (BOK)*, (July).

Pomeroy-huff, M., & Chick, T. A. (2009). *El Proceso Personal de Software SM (PSP SM)*, 1–36.

INFORMACIÓN TÉCNICA



FACULTAD DE
**INGENIERÍA, DISEÑO
E INNOVACIÓN**

Módulo: Desarrollo de *Software* en equipo

Unidad 4: Aseguramiento de la calidad

Escenario 7: Recopilación de datos y estimaciones de tiempo en proyectos de *software*

Autor: Grissa Maturana

Asesor Pedagógico: Diana Marcela Díaz Salcedo

Diseñador Gráfico: Diego Calderón

Asistente: Ginna Quiroga

Este material pertenece al Politécnico Gran Colombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumino. Prohibida su reproducción total o parcial.